ASSIGNMENT 17-3

Task 1 Employee Dataset Cleaning
Task:
Use AI assistance to clean an employee information dataset.
Instructions:
• Handle missing values in salary, designation, and experience
• Remove duplicate employee IDs
• Standardize text values (job titles in lowercase)
• Encode categorical features (designation, employment_type)
Expected Output:
A clean dataset suitable for HR analytics and prediction tasks.

[2]
0s

```python
import pandas as pd
import os

# Check if 'employee_data.csv' exists, if not, create a dummy one
if not os.path.exists("employee_data.csv"):
    print("employee_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'employee_id': [1, 2, 3, 4, 5, 6, 7],
        'salary': [50000, 60000, None, 75000, 80000, 55000, 62000],
        'designation': ['Software Engineer', 'Data Scientist', 'Manager', None, 'Software Engineer', 'Analyst', 'Marketing Specialist'],
        'experience': [3, 5, 10, None, 4, 2, 7],
        'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Full-time']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("employee_data.csv", index=False)
    print("Dummy employee_data.csv created.")

# Load dataset
df = pd.read_csv("employee_data.csv")

# ------------------------------
# 1. Handle Missing Values
# ------------------------------

# Fill missing salary with median
df['salary'].fillna(df['salary'].median(), inplace=True)
```

```python
# Fill missing designation with 'unknown'
df['designation'].fillna('unknown', inplace=True)

# Fill missing experience with median
df['experience'].fillna(df['experience'].median(), inplace=True)

# ------------------------------
# 2. Remove Duplicate Employee IDs
# ------------------------------

df.drop_duplicates(subset='employee_id', keep='first', inplace=True)

# ------------------------------
# 3. Standardize Text Values
# ------------------------------

# Convert job titles to lowercase and strip spaces
df['designation'] = df['designation'].str.lower().str.strip()

# ------------------------------
# 4. Encode Categorical Features
# ------------------------------

# One-hot encoding for categorical columns
df = pd.get_dummies(df, columns=['designation', 'employment_type'], drop_first=True)
```

```python
df.reset_index(drop=True, inplace=True)

# Save cleaned dataset
df.to_csv("cleaned_employee_data.csv", index=False)

# Preview
print(df.head())
```

```
employee_data.csv not found. Creating a dummy dataset for demonstration.
Dummy employee_data.csv created.
   employee_id   salary  experience  designation_data scientist  \
0            1  50000.0         3.0                       False
1            2  60000.0         5.0                        True
2            3  61000.0        10.0                       False
3            4  75000.0         4.5                       False
4            5  80000.0         4.0                       False

   designation_manager  designation_marketing specialist  \
0                False                             False
1                False                             False
2                 True                             False
3                False                             False
4                False                             False

   designation_software engineer  designation_unknown  \
0                           True                False
1                          False                False
2                          False                False
3                          False                 True
4                           True                False
```

EXPLANATION:

Missing values: Salary and experience are filled with median to avoid skew; designation uses `"unknown"`.

Duplicates: Ensures each `employee_id` is unique.

Standardization: Converts job titles to lowercase for consistency.

Encoding: Converts categorical variables into numeric format using one-hot encoding for ML models.

Task 2 – Student Performance Data Preprocessing

Task:

Preprocess student performance data using AI assistance.

Instructions:

• Convert exam_date column into datetime format

• Extract new features (month, semester, day)

• Normalize total_marks

• Detect and handle outliers in marks using IQR or Z-score

Expected Output:

A transformed dataset with time-based academic features and normalized marks.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import os

# Check if 'student_performance.csv' exists, if not, create a dummy one
if not os.path.exists("student_performance.csv"):
    print("student_performance.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'exam_date': pd.to_datetime(['2023-01-15', '2023-02-20', '2023-03-10', '2023-04-05', '2023-05-12']),
        'total_marks': [85, 92, 78, 65, 95, 70, 88, 55, 100, 72],
        'course': ['Math', 'Science', 'Math', 'English', 'Science', 'Math', 'English', 'Science', 'Math', 'English']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("student_performance.csv", index=False)
    print("Dummy student_performance.csv created.")

# Load dataset
df = pd.read_csv("student_performance.csv")

# ------------------------------
# 1. Convert exam_date to datetime
```

```
# ------------------------------
# 2. Extract Time-based Features
# ------------------------------
df['exam_month'] = df['exam_date'].dt.month
df['exam_day'] = df['exam_date'].dt.day

# Semester logic (example: Jan–Jun = 1, Jul–Dec = 2)
df['semester'] = df['exam_month'].apply(lambda x: 1 if x <= 6 else 2)

# ------------------------------
# 3. Normalize total_marks
# ------------------------------
scaler = MinMaxScaler()
df['total_marks_normalized'] = scaler.fit_transform(df[['total_marks']])

# ------------------------------
# 4. Detect & Handle Outliers (IQR Method)
# ------------------------------

Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Option 1: Remove outliers
df = df[(df['total_marks'] >= lower_bound) & (df['total_marks'] <= upper_bound)]
```

```python
# ------------------------------
# Final Dataset
# ------------------------------
df.reset_index(drop=True, inplace=True)

# Save cleaned dataset
df.to_csv("processed_student_data.csv", index=False)

# Preview
print(df.head())
```

```
student_performance.csv not found. Creating a dummy dataset for demonstration.
Dummy student_performance.csv created.
   student_id  exam_date  total_marks   course  exam_month  exam_day  \
0           1 2023-01-15           85     Math           1        15
1           2 2023-02-20           92  Science           2        20
2           3 2023-03-10           78     Math           3        10
3           4 2023-04-05           65  English           4         5
4           5 2023-05-12           95  Science           5        12

   semester  total_marks_normalized
0         1                0.666667
1         1                0.822222
2         1                0.511111
3         1                0.222222
4         1                0.888889
```

EXPLANATION:

Datetime conversion ensures proper time-based analysis.
Feature extraction (month, day, semester) helps identify academic patterns.
Normalization scales marks between 0–1 for ML models.
Outlier handling (IQR) removes or caps extreme values to improve model accuracy.

Task 3 – Student Health Data Preparation
Task:
Clean and preprocess student health records using AI scripts.
Instructions:
• Handle missing values in height, weight, BMI
• Encode categorical features (medical_condition, fitness_status)
• Scale numerical features using standard scaling
• Split dataset into training and testing sets
Expected Output:
A preprocessed dataset ready for predictive health analysis.

EXPLANATION:

- **Missing values** are filled with median to reduce bias.
- **Categorical encoding** converts health conditions and fitness status into numeric form.
- **Standard scaling** ensures features like height, weight, and BMI have mean = 0 and variance = 1.
- **Train-test split** prepares data for building and evaluating predictive models.

[6]
0s

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import os

# Check if 'student_health_data.csv' exists, if not, create a dummy one
if not os.path.exists("student_health_data.csv"):
    print("student_health_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'height': [170, 165, 180, 175, 160, 185, None, 172, 168, 177],
        'weight': [70, 60, 85, None, 55, 90, 75, 68, 62, 80],
        'BMI': [24.2, 22.0, 26.2, 24.5, 21.5, 26.3, 25.0, None, 22.0, 25.5],
        'medical_condition': ['None', 'Asthma', 'None', 'Diabetes', 'None', 'Asthma', 'None', 'None', 'None', 'Diabetes'],
        'fitness_status': ['Good', 'Average', 'Excellent', 'Poor', 'Good', 'Excellent', 'Average', 'Good', 'Average', 'Poor'],
        'target': [0, 1, 0, 1, 0, 0, 1, 0, 1, 0] # Assuming 'target' is a binary classification target
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("student_health_data.csv", index=False)
    print("Dummy student_health_data.csv created.")

# Load dataset
df = pd.read_csv("student_health_data.csv")

# ------------------------------
# 1. Handle Missing Values
```

[6]
0s

```python
# Fill missing numerical values with median
df['height'].fillna(df['height'].median(), inplace=True)
df['weight'].fillna(df['weight'].median(), inplace=True)
df['BMI'].fillna(df['BMI'].median(), inplace=True)

# ------------------------------
# 2. Encode Categorical Features
# ------------------------------

# One-hot encoding
df = pd.get_dummies(df, columns=['medical_condition', 'fitness_status'], drop_first=True)

# ------------------------------
# 3. Scale Numerical Features
# ------------------------------

scaler = StandardScaler()
numerical_cols = ['height', 'weight', 'BMI']

df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# ------------------------------
# 4. Split Dataset
# ------------------------------

# Assuming 'target' is the prediction column (change if needed)
X = df.drop('target', axis=1)
y = df['target']
```

[6]
0s

```python
# Final Output
# -------------------------------

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

```
student_health_data.csv not found. Creating a dummy dataset for demonstration.
Dummy student_health_data.csv created.
Training set shape: (8, 7)
Testing set shape: (2, 7)
```

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import os

# Check if 'ride_sharing_data.csv' exists, if not, create a dummy one
if not os.path.exists("ride_sharing_data.csv"):
    print("ride_sharing_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'ride_id': [1, 2, 3, 4, 5, 6, 7, 8],
        'pickup_location': ['Downtown', 'Suburb A', 'Downtown', 'Airport', 'Suburb A', 'Downtown ', '  Suburb B', 'Airport'],
        'drop_location': ['Airport', 'Downtown', 'Suburb B', 'Downtown', 'Downtown', 'Suburb C', 'Airport', 'Suburb A'],
        'fare_amount': [25.5, 15.0, None, 40.0, 18.2, 22.0, 30.0, 35.0],
        'driver_rating': [4.5, 3.8, 5.0, 4.2, 4.0, 3.5, 4.9, 4.1]
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("ride_sharing_data.csv", index=False)
    print("Dummy ride_sharing_data.csv created.")

# Load dataset
df = pd.read_csv("ride_sharing_data.csv")

# Keep a copy for summary comparison
original_df = df.copy()
```

```python
df['pickup_location'] = df['pickup_location'].str.lower().str.strip()
df['drop_location'] = df['drop_location'].str.lower().str.strip()

# ------------------------------
# 2. Handle Missing Fare Amount
# ------------------------------

df['fare_amount'].fillna(df['fare_amount'].median(), inplace=True)

# ------------------------------
# 3. Remove Duplicate Rides
# ------------------------------

df.drop_duplicates(subset='ride_id', keep='first', inplace=True)

# ------------------------------
# 4. Normalize Driver Ratings
# ------------------------------

scaler = MinMaxScaler()
df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])

# ------------------------------
# 5. Generate Summary of Improvements
# ------------------------------
```

```
ride_sharing_data.csv not found. Creating a dummy dataset for demonstration.
Dummy ride_sharing_data.csv created.
Data Cleaning Summary:
original_rows: 8
cleaned_rows: 8
duplicates_removed: 0
missing_fare_filled: 1
unique_pickup_locations_before: 5
unique_pickup_locations_after: 4
   ride_id pickup_location drop_location  fare_amount  driver_rating  \
0        1        downtown       airport         25.5            4.5
1        2        suburb a      downtown         15.0            3.8
2        3        downtown      suburb b         25.5            5.0
3        4         airport      downtown         40.0            4.2
4        5        suburb a      downtown         18.2            4.0

   driver_rating_normalized
0                  0.666667
1                  0.200000
2                  1.000000
3                  0.466667
4                  0.333333
```

EXPLANATION:

Location standardization removes inconsistencies (case, spacing).

Missing fares are replaced with median to maintain realistic values.

Duplicate rides are removed using unique `ride_id`.

Driver ratings are normalized (0–1 scale) for fair comparison.

Summary report highlights improvements made during cleaning.